

Collections and Generics

COLLECTIONS

Collections – Introduzione

Prima di tutto va chiarita una fonte di confusione terminologica. In Java esistono **tre cose diverse** che si chiamano quasi allo stesso modo:

1. **collection** (c minuscola) — è un concetto generico: qualsiasi struttura dati in cui gli oggetti vengono memorizzati e iterati.
2. **Collection** (C maiuscola) — è l'**interfaccia** al vertice della gerarchia. Le interfacce `Set`, `List` e `Queue` la estendono.
3. **Collections** (C maiuscola + S finale) — è una **classe di utilità** che contiene metodi statici da usare sulle collezioni (es. `Collections.sort`, `Collections.binarySearch`).

Collections — la classe di utilità

Collections (con la C maiuscola e la s finale) è una **classe di utilità** del package `java.util`.

Una classe di utilità è una classe composta **esclusivamente da metodi statici**, pensata per operare su altri oggetti — in questo caso, sulle collezioni. Non la istanzi mai (non fai `new`

`Collections()`); chiami direttamente i suoi metodi con la sintassi

`Collections.nomeMetodo(...)`.

Va tenuta ben distinta dalle altre due cose dal nome simile:

- **collection** (c minuscola) → il concetto generico di struttura dati.
- **Collection** (C maiuscola, senza s) → l'**interfaccia** al vertice della gerarchia, estesa da `Set`, `List`, `Queue`.
- **Collections** (C maiuscola, con la s) → la **classe** di metodi statici di cui stiamo parlando.

C'è un parallelo utile: come **Collections** opera sulle collezioni, la classe **Arrays** opera sugli array. Sono entrambe classi di utilità, entrambe derivano da **Object**.

La gerarchia semplificata è questa:

- In cima c'è **Collection**.
- Da essa derivano tre rami: **Set**, **List**, **Queue**.
- Nel ramo **Set**: `SortedSet` → `NavigableSet`. Le implementazioni sono `HashSet` (nessun ordine), `LinkedHashSet` (mantiene ordine di inserimento, sottotipo di `HashSet`), `TreeSet` (ordinato, implementa `NavigableSet`).
- Nel ramo **List**: `ArrayList` (ordine per indice), `Stack` (classe legacy, da evitare), `LinkedList`.
- Nel ramo **Queue**: `Deque` (coda a doppia estremità), `PriorityQueue` (basata su priorità). `ArrayDeque` e `LinkedList` implementano `Deque`.

Map è a parte. La Map **non** estende `Collection`, anche se se ne parla insieme alle collezioni. La sua gerarchia: `SortedMap` → `NavigableMap` → `TreeMap`; poi `HashMap` (nessun ordine), `LinkedHashMap` (ordine di inserimento), `Hashtable`.

Differenza chiave **HashMap vs Hashtable**: la `HashMap` non è thread safe, quindi è un po' più veloce della `Hashtable`, che è thread safe. Inoltre `HashMap` ammette chiavi/valori null, `Hashtable` no.

Come appaiono in memoria le quattro strutture base:

- **List**: coppie indice → valore; l'indice permette accesso diretto; **i duplicati sono ammessi** e l'ordine è mantenuto.
- **Set**: elementi **unici**, nessun duplicato.
- **Map**: coppie chiave → valore; **le chiavi non possono essere duplicate**.
- **Queue**: tipicamente **FIFO** (First In First Out), come la coda alla mensa — il primo arrivato è il primo servito. Una variante è la pila **LIFO** (Last In First Out), dove si lavora da un solo capo.

Metodi comuni della Collection (disponibili per tutto tranne Map, che non estende `Collection`): `add`, `remove` (rimuove una singola istanza), `isEmpty`, `size`, `clear`, `contains`, `removeIf` (rimuove tutti gli elementi che soddisfano una condizione), e `forEach` (metodo di default dell'interfaccia `Iterable`, esegue un'azione su ogni elemento).

Metodi di raccolta comuni nel codice

L'istruttore dichiara una collezione generica di stringhe implementata come `ArrayList` e ci mostra i metodi in azione:

```
Collection<String> coll = new ArrayList<>();
coll.add("Lucy");
coll.add("April");
coll.add("Lucy"); // duplicato ammesso: le liste li consentono
```

- `coll.remove("Lucy")` → rimuove **solo la prima occorrenza** di Lucy. Restituisce `true` perché l'ha trovata. Rimangono `April`, `Lucy`.
- `coll.isEmpty()` → `false`.
- `coll.size()` → `2`.
- `coll.contains("John")` → `false`; `coll.contains("Lucy")` → `true`.
- `coll.removeIf(s -> s.startsWith("A"))` → il metodo accetta un **predicato** (funzione che riceve un valore e ritorna `true/false`, tramite il metodo `test`). Rimuove tutte le voci che iniziano con "A" maiuscola → rimuove `April`.
- `coll.forEach(name -> System.out.println(name))` → itera passando un **consumer** che stampa ogni elemento. Si può sostituire con un **method reference**: `coll.forEach(System.out::println)`.
- `coll.clear()` → svuota la collezione.

Punto importante su `Arrays.asList` / `List.of`: se crei la collezione con questi metodi ottieni una lista **immutabile** (o a dimensione fissa). Se provi a fare `add`, ottieni una `UnsupportedOperationException`.

List

La **List** è una collezione **ordinata** (sequenza) che dà controllo preciso sull'accesso agli elementi tramite l'**indice intero** (che parte da 0). **Ammette duplicati**.

Implementazioni:

- **ArrayList**: array ridimensionabile internamente. Ottimo per **iterazione veloce e accesso casuale veloce** quando non fai molti inserimenti/cancellazioni. Il problema di inserire/cancellare è che devi spostare gli elementi su/giù per far spazio.
- **LinkedList**: gli elementi sono **doppiamente collegati** tra loro. Ottimo per inserimento/cancellazione: per eliminare un elemento basta ridirezionare il link del precedente e del successivo, escludendo quello rimosso.
- **Stack**: classe **legacy**, rappresenta una pila LIFO. Da usare al suo posto l'interfaccia `Deque` con le sue implementazioni.

Metodi factory (statici): `Arrays.asList(...)` restituisce una lista di **dimensione fissa supportata dall'array** (backed by array). Le modifiche alla lista si riflettono sull'array e viceversa; non puoi aggiungere/rimuovere ma puoi **sostituire**. `List.of(...)` e `List.copyOf(...)` restituiscono liste **immutabili**.

Metodi della List: `add(indice, elem)` inserisce a un indice spostando gli altri di un posto; `get(indice)` restituisce l'elemento; `remove(indice)` rimuove e sposta gli altri; `replaceAll operatore` applica un **UnaryOperator** (input e output dello stesso tipo) a ogni elemento; `set(indice, elem)` sostituisce l'elemento a quell'indice e restituisce quello originale.

Lists nel codice

Il principio "backed by array" (`Arrays.asList`):

```
String[] array = {"Alpha", "Beta", "Charlie"}; // riga 17
List<String> list = Arrays.asList(array);      // riga 18 – collegati
```

Ora array e lista sono **collegati**:

- `array[0] = "Delta"` → stampando l'array esce `Delta, Beta, Charlie`, ma **anche la lista** diventa `Delta, Beta, Charlie` perché condividono la memoria.
- `list.set(1, "Echo")` → cambia la seconda voce; ora **sia array che lista** mostrano `Delta, Echo, Charlie`.

Questo conferma che le modifiche passano dall'uno all'altro. Con `List.of` invece, tentare una modifica dà `UnsupportedOperationException`.

ArrayList (mantiene ordine, ammette duplicati):

```
List<String> list = new ArrayList<>();  
list.add("Alan");  
list.add("Sean");  
list.add("Alan");  
list.add("Mary");  
list.add("Mary");
```

- `list.get(1)` → Sean (indice 1 = secondo elemento).
- `list.remove(0)` → rimuove il primo (Alan). Restano Sean, Alan, Mary, Mary.
- `list.remove("Mary")` (versione overloaded con oggetto) → rimuove solo la **prima** Mary. Restano Sean, Alan, Mary.
- `list.set(0, "Jack")` → prima voce diventa Jack → Jack, Alan, Mary.
- `list.replaceAll(name -> name + " Kennedy")` → applica un UnaryOperator: Jack Kennedy, Alan Kennedy, Mary Kennedy.

Stack (LIFO, si lavora da un solo capo):

```
Stack<String> stack = new Stack<>();  
stack.push("Andrea");  
stack.push("Barbara");  
stack.push("Caroline");
```

- `stack.peek()` → Caroline (guarda la cima senza rimuoverla).
- `stack.pop()` → rimuove Caroline. Restano Andrea, Barbara.
- `stack.push("Helen")` → va in cima → Andrea, Barbara, Helen.

LinkedList (doppiamente collegata, si lavora su entrambi i capi):

```
LinkedList<String> ll = new LinkedList<>();  
ll.add("Collin");  
ll.add("David");  
ll.addFirst("Brian"); // Brian all'inizio  
ll.addLast("Eduard"); // riga 76
```

Ora: Brian, Collin, David, Eduard.

- `ll.remove("David")` → rimuove la prima occorrenza di David.
- `ll.removeFirst()` → Brian via.
- `ll.removeLast()` → Edward via. Rimane solo Collin.

Questo dimostra la differenza chiave: **LinkedList** lavora su entrambe le estremità, lo **Stack** solo su una.

Set

L'interfaccia **Set** rappresenta collezioni **senza elementi duplicati**. Implementazioni:

- **HashSet**: set **non ordinato**. Usa l'hashCode dell'oggetto inserito. Più efficiente è l'implementazione dell'hashCode, migliori le prestazioni di accesso. Usalo quando vuoi unicità e **non ti importa dell'ordine**.
 - **LinkedHashSet**: versione ordinata dell'HashSet che **mantiene l'ordine di inserimento** (elementi doppiamente collegati). Usalo quando l'ordine di iterazione conta.
 - **TreeSet**: collezione **ordinata**. Ordina secondo l'**ordine naturale** (alfabetico per stringhe, numerico per interi) oppure secondo un ordine personalizzato fornendo un **Comparator** alla creazione.
-

Set nel codice

TreeSet (unico e ordinato):

```
TreeSet<String> set = new TreeSet<>();  
set.add("John");  
set.add("John"); // duplicato ignorato  
set.add("Helen");
```

Aggiungendo John due volte, appare **una sola volta**. Stampando: ordinato alfabeticamente → Helen, John indipendentemente dall'ordine di inserimento. Un TreeSet di interi verrà ordinato **numericamente** (es. 11, 21, 23, 99).

HashSet con oggetti custom — serve parlare di hashing:

Concetto di hashing (trattato in dettaglio più avanti): **oggetti uguali devono avere lo stesso hashCode**. L'hashCode determina il "bucket" (secchio/libro) dove va l'oggetto. L'idea: non serve tenere in giro lo stesso identico oggetto per recuperarlo — puoi ricrearne uno con le **stesse variabili di istanza** e, avendo lo stesso hashCode, finirà nello stesso bucket. Fondamentale: **usa le stesse variabili di istanza sia in equals che nel calcolo di hashCode** (es. nome ed età).

```
Set<Contact> set = new HashSet<>();  
set.add(new Contact("Zoe", 45));  
set.add(new Contact("Zoe", 45)); // uguale → aggiunto una sola volta  
// Alice, Andrew, Brian, Carol...
```

Iterando con un for-each, l'ordine è **completamente casuale** — Brian esce primo pur essendo stato inserito penultimo. Nessun ordine per nome, età o inserimento.

LinkedHashSet (mantiene ordine di inserimento):

Stesso Contact, ma con `LinkedHashSet`. Aggiungendo Zoe due volte, appare una volta, ma l'output rispetta l'**ordine di inserimento**: Zoe, Alice, Andrew, Brian, Carol.

Ricapitolando i Set: vuoi unicità → `Set`. Vuoi unicità + ordinamento automatico → `TreeSet`.

Vuoi unicità + hashing senza ordine → `HashSet`. Vuoi unicità + hashing + ordine di inserimento → `LinkedHashSet`.

Maps

Le **Map** mappano **chiavi su valori**. Le chiavi sono **uniche** e ogni chiave mappa **al massimo un valore**.

Implementazioni:

- **HashMap**: non ordinata. Usa l'hashCode. Ammette **una chiave null e più valori null**.
- **LinkedHashMap**: mantiene l'**ordine di inserimento** (hashing + ordine).
- **TreeMap**: mappa **ordinata per chiavi** (ordine naturale o Comparator personalizzato).
- **Hashtable**: simile a HashMap ma **thread-safe** (quindi più lenta per la sincronizzazione); non ammette null.

Metodi della Map (più numerosi perché non eredita da Collection): `clear`;

`containsKey(obj)`; `containsValue(val)`; `entrySet()` restituisce un set di

`Map.Entry` (incapsulamento della coppia chiave-valore); `forEach` (accetta un **BiConsumer**

chiave+valore); `get(key)` (ritorna null se assente); `isEmpty`; `keySet()` (set delle chiavi);

`put(key, val)` (aggiunge o sostituisce, ritorna il valore precedente o null);

`putIfAbsent(key, val)` (aggiunge solo se assente); `remove(key)`; `replace(key,`

`val)`; `replaceAll(funzione)`; `size`; `values()` (vista di tutti i valori).

Maps nel codice

```
Map<String, Integer> map = new TreeMap<>(); // ordinata per chiave
```

```
map.put("John", 18);
```

```
map.put("Mary", 21);
```

```
map.put("Chris", 33);
```

- `map.containsKey("John")` → true.
- `map.isEmpty()` → false.
- `map.get("John")` → 18.
- `forEach` sul `keySet()` → ordine alfabetico: Chris, John, Mary.
- `map.values()` → in ordine di chiave: 33 (Chris), 18 (John), 21 (Mary).
- `map.containsKey("Paul")` → false; `map.containsValue(21)` → true.
- `map.size()` → 3; poi `clear()` → size 0.

forEach con BiConsumer:

```
map.forEach((k, v) -> System.out.println(k + " maps to " + v));
```

Il BiConsumer ha metodo funzionale `accept(T, U)` che accetta due argomenti (qui K e V) e ritorna `void`. Output ordinato per chiave: Chris → 33, John → 18, Mary → 21.

entrySet() come "ponte" verso le Collection:

```
map.entrySet().forEach(entry ->
    System.out.println(entry.getKey() + " " + entry.getValue()));
```

L'`entrySet()` converte la Map (fuori dalla gerarchia Collection) in un **Set** (dentro la gerarchia). Ogni elemento è una `Map.Entry` con `getKey()` e `getValue()`.

putIfAbsent:

```
map.putIfAbsent("Mike", null);
map.putIfAbsent("Chris", 99); // Chris esiste con 33 → NON cambia
```

La logica: se la chiave non c'è, la inserisce. Se c'è ma il **valore è null**, lo aggiorna. Se c'è con valore non-null, non fa nulla. Quindi Chris resta 33; Mike (che era null) viene aggiornato.

replaceAll con BiFunction:

```
map.replaceAll((name, age) -> name.length());
```

Sostituisce ogni valore con il risultato della funzione. La **BiFunction** ha `apply(T, U)` che prende due input e ritorna un output (qui: nome+età → lunghezza del nome). Chris → 5, John → 4, ecc. (Se cambiassi il tipo di ritorno in String otterresti errore di compilazione, perché il valore è Integer.)

- `map.remove("Mike")` → elimina Mike del tutto.

Queue and Deque

Una **Queue** specifica l'ordine di elaborazione degli elementi. Tipicamente **FIFO** (come la coda alla mensa), con eccezioni come le `PriorityQueue` (ordine naturale o `Comparator`) e le pile LIFO.

Implementazioni:

- **LinkedList** implementa Queue con le funzionalità base.
- **PriorityQueue** ordina gli elementi per priorità (ordine naturale o `Comparator`); l'elemento con priorità più alta è acceduto per primo.
- **Deque** (double-ended queue): accesso da **entrambe le estremità**. Può funzionare da coda FIFO o da pila LIFO. Implementazione: **ArrayDeque** (array ridimensionabile). L'API dice che è probabilmente più veloce di `Stack` se usato come pila e più veloce di `LinkedList` se usato come coda.

I due gruppi di metodi — punto cruciale:

Operazione	Metodi che NON lanciano eccezioni (ritornano valore speciale null/false)	Metodi che lanciano eccezioni
Esaminare la testa	<code>peek</code>	<code>element</code>
Inserire	<code>offer</code>	<code>add</code>
Rimuovere	<code>poll</code>	<code>remove</code>

Mnemonico: **peek, offer, poll** (POP per ricordarli) sono i "gentili" che non lanciano eccezioni. Per il Deque, i metodi con suffisso First/Last lavorano rispettivamente su testa/coda:

`peekFirst/offerFirst/pollFirst` e `peekLast/offerLast/pollLast`; le versioni con eccezione sono `getFirst/addFirst/removeFirst` e `getLast/addLast/removeLast`.

Queue and Deque nel codice

Queue con LinkedList (FIFO):

```
Queue<Integer> queue = new LinkedList<>();
queue.offer(1); // testa
queue.offer(2); // in fondo
queue.offer(3);
queue.offer(4);
```

La testa punta sempre a 1.

- `queue.element()` e `queue.peek()` → entrambi restituiscono 1 (guardano la testa senza rimuovere). `element` lancia eccezione se vuota, `peek` restituisce null.
- `queue.remove()` → rimuove 1 dalla testa. Ora testa = 2. (`remove` lancia se vuota, `poll` restituisce null.)
- `queue.poll()` → rimuove 2. Restano 3, 4.

ArrayDeque (entrambe le estremità):

```
Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(1);
deque.addFirst(2);
deque.addFirst(3); // testa = 3, poi 2, 1
```

- `deque.getFirst()` e `deque.peekFirst()` → 3.
- `deque.addLast(4)` e `deque.offerLast(5)` → aggiunti in coda → 3, 2, 1, 4, 5.
- `deque.removeFirst()` → toglie 3; di nuovo → toglie 2 (testa ora 1).
- `deque.removeLast()` → toglie 5; di nuovo → toglie 4. Rimane solo 1.

Mnemonico per capire quali lanciano eccezioni: i metodi che iniziano con **a** (add) o **g** (get) — come `addFirst`, `getLast` — **lanciano** eccezioni; gli altri (`peek`, `poll`, `offer`) restituiscono `null/false`.

ArrayDeque usato come Queue:

```
deque.offer(11);
deque.offer(12);
deque.peek(); // 11 (testa, non rimuove)
deque.poll(); // rimuove 11, testa ora 12
deque.poll(); // rimuove 12, coda vuota
deque.poll(); // null (nessuna eccezione)
```

PriorityQueue

La **PriorityQueue** implementa `Queue` ma **non** è FIFO/LIFO: assegna una **priorità** agli elementi, come una to-do list ("questo è più importante di quello").

Con stringhe (ordine naturale = alfabetico):

```
PriorityQueue<String> pq = new PriorityQueue<>();
pq.add("v");
pq.add("p");
pq.add("a"); // inseriti in quest'ordine
```

Usando `poll()` per estrarre, l'ordine è **alfabetico** (`a`, `p`, `v`), diverso dall'inserimento. Il metodo `hasNext` di un iteratore dice solo se c'è altro, non restituisce l'elemento.

Con interi (ordine naturale = numerico crescente):

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(11);
pq.add(5);
pq.add(2);
// poll → 2, 5, 11
```

Con priorità personalizzata (Comparator): per ordinare per un criterio non naturale, si usa un `Comparator`. Con una classe `Book` (con titolo `String` e prezzo `double` — entrambi tipi che implementano `Comparable`, per questo si usa il wrapper `Double` maiuscolo):

```
Comparator<Book> byTitle = Comparator.comparing(book -> book.getTitle());
PriorityQueue<Book> pq = new PriorityQueue<>(byTitle);
```

`Comparator.comparing` è un metodo statico che accetta una **funzione** che estrae una chiave di ordinamento **comparabile** dal tipo `T`, e restituisce un `Comparator` che ordina in base a quella chiave. Ordinando per titolo → `C++`, `Java`, `Python`. Puoi anche usare il method reference `Book::getTitle`, oppure ordinare per prezzo con `Comparator.comparing(Book::getPrice)`.

Evitare eccezioni durante l'iterazione sulle collezioni

Il problema centrale: **modificare una collezione mentre la si itera** con un for-each provoca una **ConcurrentModificationException**.

Perché l'interfaccia `Iterator` ha `remove` ma **non** `add`? Perché `remove` toglie l'elemento appena recuperato (operazione sicura e definita), mentre `add` porrebbe il problema di *dove* inserire — in un `Set` non ordinato non ha senso. La logica di inserimento è nel **ListIterator**, specifico per le liste.

Tabella riassuntiva delle strategie:

Struttura	Inserire durante iterazione	Rimuovere durante iterazione
List	<code>ListIterator.add()</code> ; per concorrenza <code>CopyOnWriteArrayList</code>	<code>Iterator.remove()</code> ; <code>removeIf()</code>
Set	inserimento differito con <code>addAll()</code> ; per concorrenza <code>CopyOnWriteArraySet</code>	<code>Iterator.remove()</code> ; <code>removeIf()</code>
Map	inserimento differito con <code>putAll()</code> ; per concorrenza <code>ConcurrentHashMap</code>	<code>Iterator.remove()</code> (sull' <code>entrySet</code>); stream con <code>filter</code>

Evitare eccezioni — nel codice

UnsupportedOperationException (diversa dalla concorrente): capita con `Arrays.asList` (dimensione fissa) o `List.of` (immutabile). Puoi **modificare gli elementi esistenti** (es. `list.set(0, "ALICE")`) ma non aggiungere/rimuovere. Soluzione: avvolgere in un nuovo `ArrayList`:

```
List<String> list = new ArrayList<>(List.of("Alice", "Bob", "Charlie")); // riga 51
```

`ConcurrentModificationException` con for-each:

```
for (String name : list) {
    if (name.equals("Bob")) {
        list.add("David"); // → ConcurrentModificationException!
    }
}
```

Soluzione 1 — ListIterator (ha add):

```
ListIterator<String> it = list.listIterator();
while (it.hasNext()) {
    String name = it.next();
    if (name.equals("Bob")) {
        it.add("David"); // add dell'iteratore, non della lista
    }
}
```

David viene aggiunto senza eccezioni: Alice, Bob, David, Charlie.

Soluzione 2 — CopyOnWriteArrayList (per concorrenza): puoi usare il normale for-each con `list.add()` direttamente, perché ogni scrittura crea una **copia separata** in memoria — niente conflitto.

Rimozione da una lista — `Iterator.remove()`:

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("Bob")) it.remove();
}
```

Rimozione — `removeIf()` (una riga, elegante):

```
list.removeIf(name -> name.equals("Bob")); // riga 101
```

È un metodo di default dell'interfaccia Collection, disponibile per tutte le implementazioni.

Set — inserimento differito:

```
Set<String> toAdd = new HashSet<>();
for (String name : names) {
    if (name.equals("Alice")) toAdd.add("Emily");
}
names.addAll(toAdd); // aggiunta dopo l'iterazione
```

Per concorrenza si usa `CopyOnWriteArraySet` col normale for-each e add diretto.

Set — rimozione: `Iterator.remove()` oppure `removeIf()`.

Map — inserimento differito con `putAll()`:

```
Map<String, Integer> toAdd = new HashMap<>();
for (Map.Entry<String, Integer> entry : names.entrySet()) { // riga 155
    if (entry.getKey().equals("Alice")) {
        // compleanno di Alice: 32
    }
}
names.putAll(toAdd); // riga 160
```

L'`entrySet()` è il "ponte" fondamentale: la Map è come un'isola (non implementa Collection); l'`entrySet` la collega alla terraferma della gerarchia Collection.

Map — per concorrenza (`ConcurrentHashMap`): puoi usare `put` direttamente nel loop, perché è costruita per non generare conflitti.

Map — rimozione con `Iterator` sull'`entrySet`:

```

Iterator<Map.Entry<String, Integer>> it = names.entrySet().iterator();
while (it.hasNext()) {
    Map.Entry<String, Integer> entry = it.next();
    if (entry.getValue() < 30) it.remove(); // rimuove chi ha meno di 30
}

```

Map — rimozione con stream + filter:

```

names = names.entrySet().stream()
    .filter(entry -> !entry.getKey().startsWith("A"))
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));

```

Converte in stream, filtra (qui elimina i nomi che iniziano per "A"), e ricolleziona in una nuova Map. Alice e Andrew vengono filtrati via.

Ordinamento – Comparable e Comparator

Questa sezione riguarda l'ordinamento e la ricerca di collezioni e array tramite le interfacce `Comparable` e `Comparator`.

Due classi di utilità diverse:

- **Collections** — classe di utilità (solo metodi statici) che opera sulle **collezioni**.
- **Arrays** — classe di utilità che opera solo sugli **array** (sintassi con parentesi quadre []).

Puoi convertire un array di tipi di riferimento in una lista con `Arrays.asList(...)`, e la lista risultante può essere passata ai metodi di `Collections`.

Le due interfacce a confronto — entrambe sono generiche, entrambe sono **interfacce funzionali** (un solo metodo astratto), entrambe servono a confrontare oggetti di tipo simile. `Comparable` è in `java.lang` (nessun import necessario); `Comparator` è in `java.util`.

Attenzione al TreeSet: se aggiungi a un `TreeSet` un oggetto la cui classe **non** implementa `Comparable`, ottieni una `ClassCastException` a **runtime**, perché il `TreeSet` si aspetta oggetti comparabili per poterli ordinare. Regola: **tutto ciò che ha "Tree" nel nome è ordinato**.

Comparable in dettaglio

Definisce **un solo metodo**: `compareTo(T o)` che ritorna un `int` e prende un solo argomento. Avendolo implementato **dentro la classe stessa**, hai già accesso alle variabili di istanza tramite `this`. Confronti quindi `this` (l'oggetto corrente) con l'oggetto passato.

`Comparable` definisce l'**ordine naturale**: crescente per gli interi (1, 2, 3...), alfabetico per le stringhe (A, B, C...). Attenzione: `TreeSet` ordina le stringhe per **Unicode** — numeri prima delle lettere, maiuscole prima delle minuscole. Mnemonico "NALL": **N**umbers, **A** (uppercase) letters, **L**owercase **L**etters.

La logica dentro `compareTo`:

- Ritorna **positivo** se l'oggetto corrente è **maggiore** di quello passato.
- Ritorna **zero** se sono equivalenti.
- Ritorna **negativo** se l'oggetto corrente è **minore**.

Puoi **delegare** questa logica a tipi esistenti come `String` e `Integer`, che hanno già implementato `Comparable`.

Coerenza con `equals`: `compareTo` ed `equals` dovrebbero essere coerenti. Secondo l'API, l'ordinamento naturale è "consistent with equals" se e solo se, per ogni coppia `E1` ed `E2`, `E1.compareTo(E2) == 0` ha lo **stesso valore booleano** di `E1.equals(E2)`. Se `compareTo` ritorna zero, `equals` dovrebbe ritornare `true`; se ritorna un valore non-zero (positivo o negativo), `equals` dovrebbe ritornare `false`. L'API **raccomanda vivamente** questa coerenza, altrimenti i `SortedSet` e le `SortedMap` "si comportano in modo strano".

Comparator in dettaglio

E se gli oggetti da ordinare **non fossero comparabili**, o se volessi ordinare in **modi diversi**? Entra in gioco il `Comparator`. Mentre con `Comparable` scegli **una** variabile di istanza come ordine naturale, con `Comparator` puoi ordinare per una variabile diversa.

È anch'essa un'interfaccia funzionale, con il metodo astratto `compare(T o1, T o2)` che accetta **due** oggetti. La logica interna è la stessa di `compareTo`, ma tipicamente il codice sta **esternamente** alla classe di cui confronti gli oggetti (mentre in `Comparable` è interno alla classe).

Come ricordare la differenza (mnemonici dell'istruttore):

- **Comparator** accetta **due** argomenti → "compa**r**a**T**or" ha il metodo `compare` (due argomenti). Le ultime due lettere dell'interfaccia (o + r) e l'ultima lettera del metodo astratto: **R** per Comparato**R**, **E** per compar**E**.
 - **Comparable** accetta **un** argomento (quindi ordine naturale) → il metodo è `compareTo`. Interfaccia comparab**l**e (**le**), metodo `compareTo` (**o**) → "Leo". `Comparable` = Leo.
-

La ricerca binaria

La **binary search** richiede un elenco **già ordinato** (come `sort`). Se non vuoi l'ordine naturale, puoi passare un `Comparator`. È un algoritmo **divide et impera**: prende il punto centrale della lista ordinata, controlla se la chiave è nella metà inferiore o superiore, poi dimezza ricorsivamente lo spazio di ricerca finché non trova l'elemento.

Code: Comparable – Product

Una classe `Product` con una variabile di istanza privata `id` (intero):

```
class Product implements Comparable<Product> {  
    private int id;  
    public Product(int id) { this.id = id; }  
    public int getId() { return id; }  
  
    @Override public int hashCode() { /* usa id */ }  
    @Override public boolean equals(Object o) { /* confronta id */ }  
  
    @Override  
    public int compareTo(Product product) {  
        return Integer.compare(this.id, product.getId());  
        // in alternativa: return this.id - product.getId(); (crescente per id)  
    }  
}
```

Nota: `compareTo` è **coerente con equals** (entrambi basati su `id`). Delega a `Integer.compare` perché `Integer` implementa già `Comparable`. La versione `id - product.getId()` ordina crescente: se l'id corrente è maggiore restituisce positivo, se minore negativo, se uguale zero.

Programma driver:

```
List<Product> products = new ArrayList<>(); // riga 12  
// aggiungo prodotti in ordine NON crescente per id  
Collections.sort(products);
```

Stampando la lista, è ordinata per **id numerico crescente** → es. 9, 19, 99.

Code: Comparable and Comparator – Sorting Arrays and Lists – Dog

Una classe `Dog` con `name` (String) e `age`. Implementa `Comparable` con **ordine naturale per nome**. Vogliamo poter ordinare anche per età.

Comparable (ordine naturale = nome), su array e su lista (metodo overloaded):

```
Dog[] dogs = { new Dog("Spot", 2), new Dog("Rover", 7) };  
Arrays.sort(dogs);  
// ordinamento alfabetico → Rover primo, Spot secondo (era Spot, Rover)
```

```
List<Dog> list = Arrays.asList(new Dog("Spot", 2), new Dog("Rover", 7)); // riga 21
Collections.sort(list);
// stesso risultato: Rover primo, Spot secondo
```

Comparator (ordinare per età invece che per nome):

// Su array:

```
Comparator<Dog> byAge = Comparator.comparing(dog -> dog.getAge());
Arrays.sort(dogs, byAge);
// esce in ordine di età crescente
```

// Su lista, con method reference e inverso:

```
Comparator<Dog> byAge2 = Comparator.comparing(Dog::getAge);
Collections.sort(list, byAge2);           // crescente per età
Collections.sort(list, byAge2.reversed()); // decrescente per età
```

`Comparator.comparing` ha bisogno di una **funzione** (accetta un parametro, ritorna un valore). Essendo generico su `Dog`, l'input è un `Dog` e ne estrai l'età. Con `Dog::getAge` è una scorciatoia — Java costruisce il `Comparator` dietro le quinte. `.reversed()` inverte l'ordine.

Code: Sorting by Multiple Fields

Una classe `Cat` che **non definisce alcun ordine naturale** (nessun `implements Comparable`, nessun `compareTo`). Vuoi ordinare i gatti per **nome**, ma se i nomi sono **identici**, come decidi chi va prima? Ordini per **età** come criterio secondario:

```
Comparator<Cat> comparator = Comparator
    .comparing(Cat::getName)      // criterio primario: nome
    .thenComparing(Cat::getAge);  // criterio secondario: età

Collections.sort(cats, comparator);
```

Con `Bella(2)`, `Bella(7)`, `Trixie(5)`: prima si ordina per nome (le due `Bella` prima di `Trixie`), poi tra le due `Bella` si usa l'età → output: **`Bella(2)`, `Bella(7)`, `Trixie(5)`**.

Code: `TreeSet Comparable` issue – come risolvere con `Comparator`

Il problema: un `TreeSet` lancia `ClassCastException` se la classe non è `Comparable`. Ecco due classi:

- Worker: ha un `id`, ma **non** implementa `Comparable` → nessun ordine naturale.
- Boss: implementa `Comparable`, `compareTo` ordina per `id` crescente.

Con Boss (funziona):

```
TreeSet<Boss> bosses = new TreeSet<>();
```

// aggiungo boss con id 20, 10, 15

Il `TreeSet` è **sempre ordinato mano a mano che aggiungi** (con un piccolo overhead di prestazioni). Non serve chiamare `Collections.sort` — l'output esce già 10, 15, 20.

Con Worker (problema):

```
TreeSet<Worker> workers = new TreeSet<>(); // costruttore senza argomenti
```

```
workers.add(new Worker(...)); // → ClassCastException: Worker non è Comparable
```

Soluzioni — passare un Comparator al costruttore del TreeSet:

// 1. Lambda esplicita

```
TreeSet<Worker> workers = new TreeSet<>(
    (w1, w2) -> w1.getId() - w2.getId());
```

// 2. Comparator.comparing con lambda (key extractor)

```
TreeSet<Worker> workers = new TreeSet<>(
    Comparator.comparing(w -> w.getId()));
```

// 3. Comparator.comparing con method reference (unbound)

```
TreeSet<Worker> workers = new TreeSet<>(
    Comparator.comparing(Worker::getId));
```

Il `compare` del `Comparator` prende due argomenti (i due worker) e ritorna negativo/zero/positivo in base al confronto degli `id`. Tutte e tre le versioni ordinano i worker per `id` (es. 10, 20, 21, 30).

Code: Binary Search

Una classe `Cat` che implementa `Comparable` con **ordine naturale per nome crescente**, con `equals` **coerente** con `compareTo` (se i nomi danno zero → `equals` è true; se non-zero → `equals` è false).

Ricerca su stringhe:

```
List<String> list = new ArrayList<>(); // John, Martin, Paula, Anne (non ordinati)
```

```
Collections.sort(list); // OBBLIGATORIO prima della binary search
```



```
// ora: Anne, John, Martin, Paula
```

```
Collections.binarySearch(list, "John"); // → indice 1 (trovato)
```

```
Collections.binarySearch(list, "Laura"); // → -3 (non trovato)
```

Il valore negativo per l'elemento **non trovato**: la binary search calcola dove *apparirebbe* Laura nella lista ordinata (all'indice 2, tra John e Martin), poi restituisce - (punto di inserimento) - 1 = - (2) - 1 = -3.

Ricerca con tipo custom e ordine naturale:

```
List<Cat> cats = Arrays.asList(new Cat("Fluffy", ...), new Cat("Bella", ...));
```

```
Collections.sort(cats); // per nome → Bella, Fluffy
```

```
Collections.binarySearch(cats, bella); // → 0
```

L'API avverte: la lista **deve** essere ordinata secondo l'ordine naturale prima della chiamata; altrimenti **i risultati sono indefiniti**. Se ci sono elementi equivalenti, non è garantito quale venga trovato.

Ricerca con Comparator:

```
Comparator<Cat> byAge = Comparator.comparing(Cat::getAge);
```

```
Collections.sort(cats, byAge); // deve essere ordinato con LO STESSO comparator
```

```
Collections.binarySearch(cats, bella, byAge); // → indice 1
```

Regola: se ordini con un Comparator, devi cercare con lo **stesso** Comparator, e la lista deve essere ordinata crescente secondo quel Comparator.

GENERICIS

Generics – Introduzione

I generics sono stati introdotti in **Java 1.5**. Gli array in Java sono sempre stati **tipizzati**: un array di stringhe non accetta interi o cani.

Prima dei generics (collezioni "grezze", raw): si usava la classe `Object` per memorizzare gli elementi. Questo porta a problemi di **type safety non rilevabili dal compilatore**, che emergono solo a **runtime**. Un contenitore di `Object` ha una debolezza critica: **perdi l'informazione sul tipo**. Devi fare un **cast** quando recuperi gli elementi, e il compilatore non può determinare se il cast è corretto → `ClassCastException` a runtime.

Con i generics specifici al compilatore che solo elementi di un certo tipo possono essere aggiunti. Ogni tentativo di aggiungere un tipo diverso viene bloccato a **tempo di compilazione** — questa è la **type safety**. Inoltre i generics permettono di scrivere codice per un tipo applicabile a tutti i tipi, invece di scrivere classi separate. Il tipo generico è quello tra le **parentesi angolari** `<>`.

Pre-generics vs generics nel codice:

// PRE-GENERICs (raw) — pericoloso:

```
List myList = new ArrayList();  
myList.add("Fred");    // String  
myList.add(new Dog());  // Dog  
myList.add(43);         // Integer (autoboxing) — tutto trattato come Object
```

```
String s = (String) myList.get(0); // ok, cast necessario
```

```
String s2 = (String) myList.get(1); // → ClassCastException a runtime!
```

// GENERICs — sicuro:

```
List<String> list = new ArrayList<String>();
```

// oppure con type inference (diamond operator):

```
List<String> list = new ArrayList<>();
```

```
list.add("Fred");    // ok
```

```
list.add(new Dog()); // → ERRORE DI COMPILAZIONE
```

```
String s = list.get(0); // nessun cast necessario
```

L'**operatore diamante** <> (type inference) permette di omettere il tipo a destra, dedotto dal lato sinistro.

Type Erasure (cancellazione del tipo)

Cos'è? Per rendere il codice generico **compatibile** con il vecchio codice pre-generics (milioni di righe già esistenti), gli ingegneri Java decisero di **rimuovere tutte le informazioni sul tipo dal bytecode**. Questo significa che una `List<String>` è, a runtime, **identica** a una `List` non generica: la JVM vede lo stesso bytecode, senza informazioni di tipo.

Conseguenza fondamentale: i generics sono **strettamente una protezione a tempo di compilazione**. A runtime non esistono più.

Generics – polimorfismo, extends, super

Il polimorfismo funziona sul tipo base, ma NON sul tipo generico.

```
List<Integer> list = new ArrayList<>(); // OK
```

Anche se `Integer` è sottotipo di `Number`, questo **non** è permesso.

Perché? Immagina fosse consentito:

```
List<Double> doubles = new ArrayList<>();
```

```
doubles.add(12.3);
```

```
List<Object> objects = doubles; // SE fosse permesso...
```

```
objects.add("una stringa"); // ...ora una String è dentro una lista di Double!
```

Questo comprometterebbe la type safety, quindi il compilatore blocca l'assegnazione. Stesso problema passando una `List<String>` a un metodo che si aspetta `List<Object>`.

La soluzione: i wildcard ?

Esistono tre tipi:

1. Unbounded wildcard — ?:

```
List<?> list = new LinkedList<Integer>(); // OK
```

2. Upper bound wildcard — ? extends Tipo:

```
List<? extends Number> list = new LinkedList<Integer>(); // OK
```

3. Lower bound wildcard — ? super Tipo:

```
List<? super Number> list = new ArrayList<Object>(); // OK
```

Regola cruciale su lettura/scrittura:

- All'**unbounded** e all'**upper bound** (extends) **non puoi aggiungere** elementi → sono di **sola lettura** (immutabili in scrittura).
- Al **lower bound** (super) **puoi aggiungere** elementi alla collezione.

Unbounded nel codice: un `List<?>` può riferirsi a `new ArrayList<String>()`, `new ArrayList<Dog>()`, `new ArrayList<Cat>()` — qualsiasi cosa. Ma è di sola lettura.

Bounded wildcards limitano i tipi utilizzabili, in entrambe le direzioni:

- `? extends Number` (**upper bound**): il tipo sconosciuto è `Number` o un suo **sottotipo** (`Integer`, `Double`...). "extends" qui è usato in senso generale — vale sia per classi (extends) che per interfacce (implements). Sola lettura.
- `? super Integer` (**lower bound**): il tipo sconosciuto è `Integer` o un suo **supertipo**. Puoi aggiungere in sicurezza.

Generics – super and extends (spiegazione dettagliata)

Gerarchia usata: `Animal` → `Dog` → `Terrier`; `Animal` → `Cat` → `Manx` (chiamato "Max"/"Manx").

Ripasso del problema del polimorfismo: array e generics hanno lo stesso problema con gli assegnamenti polimorfici, ma il **comportamento differisce**. A causa della type erasure, la JVM **non conosce i tipi generici**, mentre **conosce** i tipi degli array.

```
// ARRAY — la JVM conosce il tipo:
```

```
Dog[] dogs = { new Dog(), new Dog() };
```

```
Animal[] animals = dogs; // permesso (polimorfismo array)
animals[0] = new Dog(); // ok
animals[0] = new Cat(); // → ArrayStoreException a RUNTIME
```

La JVM sa che dovrebbero esserci Dog, quindi lancia `ArrayStoreException`. Con i generics questo controllo runtime non esiste, quindi il **compilatore** deve intervenire e bloccare tutto in anticipo — ecco perché `List<Animal> = new ArrayList<Cat>()` è un errore di **compilazione**.

Polimorfismo sul tipo base va bene:

```
List<Cat> list = new ArrayList<Cat>(); // OK: List/ArrayList è polimorfismo del tipo base
List<Animal> a = new ArrayList<Cat>(); // ERRORE: tipo generico non corrisponde
```

extends – flessibilità con lettura (PECS: "Producer Extends"):

```
List<? extends Animal> list = new ArrayList<Animal>();
// può riferirsi a liste di Animal, Dog, Terrier, Cat, Manx — qualsiasi cosa da Animal in giù
```

`Animal` è il **limite superiore** (upper bound). Puoi passare al metodo liste di `Animal`, `Cat`, `Manx`, `Dog`, `Terrier`. Ma è **sola lettura**:

```
void metodo(List<? extends Animal> list) {
    list.add(new Dog()); // → ERRORE DI COMPILAZIONE
    Animal a = list.get(0); // OK: qualunque cosa sia, è un Animal
}
```

Perché non puoi aggiungere? Se fosse permesso, potresti passare una `List<Cat>` e aggiungerci un `Dog` — violando la type safety. In lettura invece è sicuro: qualunque tipo concreto sia, è comunque un `Animal`, quindi lo leggi come `Animal`. Non puoi però leggerlo come `Dog` (potrebbe essere una lista di `Cat`).

super – flessibilità con scrittura (PECS: "Consumer Super"):

```
List<? super Cat> list = new ArrayList<Object>();
// può riferirsi a liste di Cat, Animal, Object — Cat e i suoi supertipi
```

`Cat` è il **limite inferiore** (lower bound). Questo è **modificabile**:

```
void metodo(List<? super Cat> list) {
    list.add(new Cat()); // OK: un Cat è sicuro ovunque
    list.add(new Manx()); // OK: Manx è un Cat
    list.add(new Dog()); // ERRORE: Dog non è un Cat né un suo sottotipo
}
```

Perché puoi aggiungere Cat? Il metodo potrebbe ricevere una lista di `Cat`, `Animal` o `Object`. In tutti i casi un `Cat` "ci sta": un `Cat` è un `Cat`, un `Cat` è un `Animal`, un `Cat` è un `Object`. Quindi

aggiungere un `Cat` (o un suo sottotipo come `Manx`) è sempre sicuro. Non puoi aggiungere `Dog/Animal/Object` perché la lista potrebbe essere una `List<Cat>` che non li accetta.

Cosa puoi leggere da `super`? Solo come `Object`: la lista potrebbe essere di `Animal` o `Object`, quindi l'unica garanzia sicura è che ogni elemento è un `Object`.

Mnemonic PECS: **P**roducer **E**xtends, **C**onsumer **S**uper. Se la collezione *produce* dati (li leggi) → `extends`. Se *consuma* dati (ci scrivi) → `super`. Altro trucco dell'istruttore: "**E** per **E** = **E**xtends = **r**ead-only".

Generic Classes and Interfaces

Puoi creare tipi generici personalizzati. La sintassi dichiara un **parametro di tipo formale** tra parentesi angolari. Esempio dall'API: l'interfaccia `List<E>` definisce `E`, che poi appare nei metodi (es. `boolean add(E e)`). `E` è un **segnaposto** per il tipo che passerai; dichiarando `List<String>`, `String` sostituisce `E`.

Classe generica personalizzata:

```
class MyGeneric<T> {
    private T t;
    public MyGeneric(T t) { this.t = t; }
    public T getT() { return t; }
}

MyGeneric<String> l = new MyGeneric<>("SK"); // T = String
l.getT(); // → "SK"

MyGeneric<Integer> i = new MyGeneric<>(1); // T = Integer (autoboxing)
i.getT(); // → 1
```

Il potere: una sola classe funziona per **qualsiasi** tipo — non devi scrivere una classe per `String`, una per `Integer`, ecc. A destra usi il diamante `<>` (type inference).

Convenzioni di denominazione dei parametri di tipo formale (singole lettere maiuscole):

- **E** = Element
- **T** = Type (tipo generico)
- **K** = Key (chiave della mappa)
- **V** = Value (valore della mappa)
- **N** = Number
- **S, U, V** = tipi generici multipli

Classe con più tipi:

```
class Register<T, U, V> {
```

```

private T type;
private U name;
private V age;
public Register(T type, U name, V age) {
    this.type = type; this.name = name; this.age = age;
}
public T getType() { return type; }
public U getName() { return name; }
public V getAge() { return age; }
}
Register<Dog, String, Integer> r1 = new Register<>(new Dog(), "Rex", 3);
Register<Cat, String, Integer> r2 = new Register<>(new Cat(), "Whisker", 2);

```

Interfacce generiche:

```

interface Movable<T> {
    void move(T t);
}

// Modo 1: implementazione fissa il tipo
class MoveFeline implements Movable<Cat> {
    public void move(Cat cat) { ... } // move accetta Cat
}

class MoveCanine implements Movable<Dog> {
    public void move(Dog dog) { ... } // move accetta Dog
}

// Modo 2: la classe stessa resta generica (più flessibile)
class SomeMove<T> implements Movable<T> {
    public void move(T t) { ... }
}

```

Con `MoveFeline`, `move` accetta solo `Cat` (errore se passi un `Dog`). Con `SomeMove<Dog>`, `move` accetta `Dog`; con `SomeMove<Cat>`, accetta `Cat` — più flessibile.

Generic Methods

I generics si possono usare anche **sui singoli metodi**. Il marcatore generico (parametro di tipo formale) si dichiara **subito prima del tipo di ritorno**. Il tipo di ritorno può anche incorporare il marcatore (rende il codice più difficile da leggere).

Primo metodo generico (ritorno void):

```
public <T> void firstMethod(T t) {  
    MyGeneric<T> mg = new MyGeneric<>(t);  
    mg.getT();  
}
```

Il <T> sta prima di void. La T nella lista parametri non dà problemi.

```
firstMethod("SK"); // T = String, t = "SK"
```

```
firstMethod(1.1); // T = Double, t = 1.1
```

Metodo generico con più tipi:

```
public <T, U, V> void register(T t, U u, V v) {  
    Register<T, U, V> r = new Register<>(t, u, v);  
    r.getType(); r.getName(); r.getAge();  
}  
  
register(new Dog(), "Ship", 3); // T=Dog, U=String, V=Integer  
register(new Cat(), "Whisker", 2); // T=Cat, U=String, V=Integer
```

Metodo generico con tipo di ritorno generico:7

```
public <T> MyGeneric<T> createGeneric(T t) {  
    return new MyGeneric<>(t);  
}  
  
MyGeneric<Integer> myGen = createGeneric(4); // T = Integer, t = 4  
myGen.getT(); // → 4
```

Qui il marcatore <T> sta prima del tipo di ritorno MyGeneric<T>.

equals()

L'operatore == confronta i **riferimenti** agli oggetti: valuta true solo quando entrambi i riferimenti puntano allo **stesso oggetto** (confronta i bit delle variabili di riferimento). Il metodo Object.equals() di default fa **esattamente la stessa cosa** (usa == internamente).

```
Person p1 = new Person();
```

```
Person p2 = new Person();
```

// p1 e p2 si riferiscono a due oggetti DIVERSI → p1 == p2 è false

La classe **String** ha sovrascritto `equals` per confrontare se due oggetti `String` sono **significativamente equivalenti** (confronta i **contenuti**, non i riferimenti).

Quando usare cosa:

- Sapere se due riferimenti sono **identici** → usa `==`.
- Sapere se due **oggetti sono uguali** (nel contenuto) → usa `equals`.

Perché sovrascrivere `equals` è importante: se memorizzi oggetti come chiavi in una `HashMap/Set` e poi vuoi recuperarli, senza override di `equals` la ricerca usa `==` sui riferimenti. Quindi, a meno di avere **esattamente** il riferimento originale (che potrebbe essere fuori scope), non ritrovi l'oggetto. Se invece sovrascrivi `equals`, puoi **ricreare** una nuova istanza "significativamente equivalente" e usarla per la ricerca, senza preoccuparti di rintracciare l'istanza originale.

Firma corretta e implementazione sicura:

```
class Foo {  
    private int foo;  
    public Foo(int foo) { this.foo = foo; }  
    public int getFoo() { return foo; }  
  
    @Override  
    public boolean equals(Object obj) {        // deve prendere Object!  
        if (!(obj instanceof Foo)) return false; // controllo sicuro prima del cast  
        return ((Foo) obj).getFoo() == this.foo; // confronta i valori  
    }  
}
```

Punti chiave:

- Usa **`instanceof`** per verificare il tipo prima del downcast, altrimenti il cast fallirebbe.
- L'**AND logico corto-circuito** (`&&`) protegge: se `!(obj instanceof Foo)` è già false a sinistra... in realtà l'istruttore mostra il pattern in cui, passando una `String`, `instanceof` fallisce e il corto-circuito evita la `ClassCastException`.

```
Foo f1 = new Foo(10);
```

```
Foo f2 = new Foo(10);
```

```
f1.equals(f2); // → true (stesso valore foo)
```

```
f1.equals("SK"); // → false (instanceof fallisce, corto-circuito, niente eccezione)
```

Override illegal (da esame di certificazione):

- `equals` **deve essere public** (String equals e hashCode sono entrambi public).
 - Il parametro deve essere **Object**, non `Foo`. Scrivere `boolean equals(Foo obj)` è un **overload**, non un override.
-

hashCode()

Il contratto: se due oggetti sono uguali secondo `equals`, **devono avere valori di hashCode identici**. Se sovrascrivi `equals`, **devi** sovrascrivere anche `hashCode`.

A cosa serve l'hashCode: migliora le prestazioni delle collezioni basate su hash (HashSet, HashMap). Il valore hashCode determina in quale **bucket** ("secchio"/"libro") l'oggetto viene memorizzato, e poi lo stesso hashCode viene usato per **localizzare** l'oggetto.

L'analogia dei secchi (bucketing):

- Un unico secchio gigante con tutti gli oggetti → devi cercare uno per uno, inefficiente.
- Molti secchi piccoli, idealmente un oggetto ciascuno → massima efficienza.

L'hashCode ti dice **in quale secchio andare**. Ma gli hashCode **non sono necessariamente unici** — oggetti diversi possono finire nello stesso secchio (collisione). A quel punto entra in gioco `equals` per trovare l'oggetto **corretto** dentro il secchio.

Il flusso di ricerca: hashCode → trova il secchio → equals → trova l'elemento esatto. Per localizzare un oggetto, l'oggetto di ricerca e quello memorizzato devono avere lo **stesso hashCode** (stesso secchio) e `equals` deve ritornare `true`.

Esempio dell'algoritmo di hashing (illustrativo): sommando valori delle lettere, "Bob" → 19, "Amy" e "I" → 39 ciascuno. Amy e I finiscono nello stesso secchio (39); Bob è da solo nel secchio 19. Cercando "I": vai al secchio 39, trovi prima Amy (equals → no), poi I (equals → sì).

Punti critici:

- Il metodo `hashCode()` di default di `Object` ritorna un numero (quasi) unico per ogni oggetto. Se sovrascrivi solo `equals` dicendo che due oggetti sono uguali, ma **non** allinei l'hashCode, finiscono in **secchi diversi** e non li ritrovi.
- Nel calcolo di hashCode usa le **stesse variabili di istanza** usate in `equals`. Così oggetti uguali per equals avranno lo stesso hashCode.
- **Non usare variabili transient** nel calcolo dell'hashCode: non vengono serializzate. Se serializzi l'oggetto su disco, al ripristino la variabile transient ha il valore di default (0 per i numeri), quindi l'hashCode ricalcolato sarà **diverso** e finiresti nel secchio sbagliato.

Esempio:

```
class Foo {  
    private int foo;  
    // ... equals basato su foo ...  
    @Override  
    public int hashCode() {
```

```
    return foo * 17; // stessa variabile usata in equals
}
}
```

Nota: un `hashCode()` che ritorna sempre 100 è **tecnicamente corretto** (rispetta il contratto) ma **terribilmente inefficiente**, perché ogni oggetto finisce nello stesso secchio anche se diverso. Non è una violazione del contratto, solo pessime prestazioni.

Riassunto del contratto: due oggetti uguali devono avere lo stesso `hashCode`. Firma: `public int hashCode()`. Per usare i tuoi oggetti in collezioni basate su hashing devi sovrascrivere **sia** `equals` (trova l'oggetto nel secchio) **sia** `hashCode` (trova il secchio), usando le **stesse variabili di istanza** in entrambi.

Hashing in code

Una classe `Contact` con `name` ed `age`, che usa **entrambe** le variabili in `hashCode` (età direttamente + lunghezza del nome) ed `equals`:

```
class Contact {
    private String name;
    private int age;

    @Override
    public int hashCode() {
        return age + name.length(); // usa entrambe le variabili
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Contact)) return false;
        Contact c = (Contact) obj;
        return this.age == c.age && this.name.equals(c.name);
    }
}
```

Mappa statica di classe:

```
static Map<Contact, String> map = new HashMap<>();
```

// in main:

```
map.put(new Contact("John", 33), "Irlandese");
map.put(new Contact("Peter", 34), "Americano");
```

Recuperare `map.get(john)`: prima calcola l'hashCode (trova il secchio), poi `equals` (trova John dentro il secchio) → ritorna "Irlandese".

Il punto forte — ricreare l'oggetto in un altro scope:

// In main creo Mary e la inserisco:

```
Contact mary = new Contact("Mary", 21); // hashCode 5732
map.put(mary, "Engineer");
```

// In un ALTRO metodo/scope, Mary locale è fuori portata.

// Ma posso RICREARE una Mary equivalente:

```
Contact anotherMary = new Contact("Mary", 21); // stesso hashCode 5732
// anotherMary accede alla STESSA voce della mappa:
map.put(anotherMary, "Accountant"); // sovrascrive Engineer
```

Poiché `anotherMary` ha lo stesso hashCode (stesso secchio) e `equals` ritorna true, mappa alla **stessa voce**. La mappa è accessibile (variabile statica di classe), Mary locale no — ma ricreando un oggetto con le **stesse variabili di istanza** la ritrovi. Tornando in main, `map.get(mary)` ora restituisce "Accountant".

Collisione con equals che distingue:

// Jane, 21 anni → stesso hashCode di Mary (stesso secchio!)

```
map.put(new Contact("Jane", 21), "Nurse");
```

Jane finisce nello **stesso secchio** di Mary (stesso hashCode). Ma `equals` li distingue: cercando Jane, vai al secchio, `equals` con Mary → no, `equals` con Jane → sì. Non c'è garanzia sull'ordine dentro il secchio (le HashMap non garantiscono ordine), ma `equals` trova sempre quello giusto. Output finale della mappa: John(33) → Irlandese, Mary(21) → Accountant, Jane(21) → Nurse, Peter(34) → Americano.

Watch out for mutable fields in hashing

Questo è un avvertimento cruciale: **se usi campi mutabili nel calcolo dell'hashCode, attenzione.**

```
class Dog {
    private String name; // usato in equals E in hashCode
    // equals: confronta i nomi
    // hashCode: basato sulla LUNGHEZZA del nome
}
Dog spot = new Dog("Spot");
Map<Dog, String> map = new HashMap<>();
map.put(spot, "Great Dane");

spot.hashCode(); // es. 9x
map.containsKey(spot); // → true (Spot presente come chiave)
```

L'API avverte: bisogna prestare molta attenzione se si usano oggetti mutabili come chiavi di mappa. Il comportamento della mappa **non è specificato** se il valore di un oggetto viene modificato in modo da influenzare i confronti di uguaglianza mentre l'oggetto è una chiave.

// Cambio lo STATO dell'oggetto — il nome usato in hashCode/equals:

```
spot.setName("Rex"); // la lunghezza del nome è cambiata!
```

```
spot.hashCode(); // ora es. 8x (DIVERSO)
```

```
map.containsKey(spot); // → FALSE!
```

Cambiando il nome cambia l'hashCode, quindi ora vai a cercare in un **secchio diverso** — un secchio che non contiene nulla. L'oggetto è ancora nella mappa (nel vecchio secchio), ma non lo ritrovi più. **Morale: non usare campi mutabili nel calcolo dell'hashCode**, o non modificarli mentre l'oggetto è chiave in una collezione basata su hash.